SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1– Experiments

Course Code:	ITA05	**Course Title:**	Computer Vision
CO Assessed:	CO3 – Precision (accurate feature extraction & analysis) (BL3)	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	**Total Marks:**	100
CO3 Statement:	*Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)*		
Student Name:	*S. Susannal Devapriyam*	**Reg. No.:**	*192525398*
Section / Branch:	B. Tech AIML	**Date:**	12-08-2026

Code of Conduct

I, S. Susannal Devapriyam (192525398) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *S. Susannal Devapriyam*

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

9. Preprocessing vs Feature Extraction

Conduct a comparative study of feature extraction performed with and without preprocessing. Explain the relationship between preprocessing and feature extraction performance. Execute both approaches using suitable tools, document the observations systematically, and analyze the differences in feature quality, detection accuracy, and overall system performance.

Aim :

To compare feature extraction with and without preprocessing and analyze their effect on feature quality and classification accuracy.

Tools Required

- Python
- OpenCV
- NumPy
- Scikit-image
- Scikit-learn
- Matplotlib

Dataset

Scikit-learn Digits Dataset – handwritten digits from 0–9.

Theory :

Preprocessing improves an image before feature extraction. It may include resizing, noise removal, contrast enhancement and normalization.

Feature extraction converts an image into useful numerical features. In this experiment, **HOG (Histogram of Oriented Gradients)** is used to extract edge and shape information.

Two methods are compared:

1. Without Preprocessing

Image → HOG → SVM → Accuracy

2. With Preprocessing

   Image → Resize → Gaussian Filter → Histogram Equalization → Normalization → HOG → SVM → Accuracy

Procedure :

1. Load the digits dataset.
2. Divide the dataset into training and testing sets.
3. Extract HOG features directly from the original images.
4. Train an SVM classifier and calculate accuracy.
5. Resize and preprocess the same images.
6. Extract HOG features from the preprocessed images.
7. Train another SVM classifier.
8. Calculate and compare both accuracies.
9. Display the original/preprocessed images and comparison graph.

Program :

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from skimage.feature import hog
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

digits = load_digits()

X_train, X_test, y_train, y_test = train_test_split(
    digits.images,
    digits.target,
    test_size=0.2,
    random_state=42,
    stratify=digits.target
)
```

```
# Without preprocessing
def get_hog(images):
    features = []

    for image in images:
        feature = hog(
            image,
            orientations=9,
            pixels_per_cell=(2, 2),
            cells_per_block=(2, 2),
            block_norm='L2-Hys'
        )
        features.append(feature)

    return np.array(features)


X1_train = get_hog(X_train)
X1_test = get_hog(X_test)

model1 = SVC(kernel='rbf', C=10)
model1.fit(X1_train, y_train)

pred1 = model1.predict(X1_test)
acc1 = accuracy_score(y_test, pred1)


# With preprocessing
def preprocess(image):

    image = image.astype(np.uint8)

    image = cv2.resize(
        image,
        (32, 32),
        interpolation=cv2.INTER_CUBIC
    )

    image = cv2.GaussianBlur(image, (3, 3), 0)
    image = cv2.equalizeHist(image)
    image = image / 255.0

    return image
```

```
def get_processed_hog(images):
    features = []

    for image in images:
        image = preprocess(image)

        feature = hog(
            image,
            orientations=9,
            pixels_per_cell=(4, 4),
            cells_per_block=(2, 2),
            block_norm='L2-Hys'
        )

        features.append(feature)

    return np.array(features)


X2_train = get_processed_hog(X_train)
X2_test = get_processed_hog(X_test)

model2 = SVC(kernel='rbf', C=10)
model2.fit(X2_train, y_train)

pred2 = model2.predict(X2_test)
acc2 = accuracy_score(y_test, pred2)


# Display results
print("WITHOUT PREPROCESSING")
print("Feature Size:", X1_train.shape[1])
print("Accuracy:", round(acc1 * 100, 2), "%")

print("\nWITH PREPROCESSING")
print("Feature Size:", X2_train.shape[1])
print("Accuracy:", round(acc2 * 100, 2), "%")

print("\nImprovement:",
      round((acc2 - acc1) * 100, 2),
      "percentage points")
```

```
# Display images
plt.figure(figsize=(7, 3))

plt.subplot(1, 2, 1)
plt.imshow(X_test[0], cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(preprocess(X_test[0]), cmap="gray")
plt.title("Preprocessed")
plt.axis("off")

plt.show()


# Accuracy graph
plt.bar(
    ["Without Preprocessing", "With Preprocessing"],
    [acc1 * 100, acc2 * 100]
)

plt.ylabel("Accuracy (%)")
plt.title("Accuracy Comparison")
plt.show()
```

Output :

WITHOUT PREPROCESSING

Feature Size: 324

Accuracy: 97.5 %

WITH PREPROCESSING

Feature Size: 1764

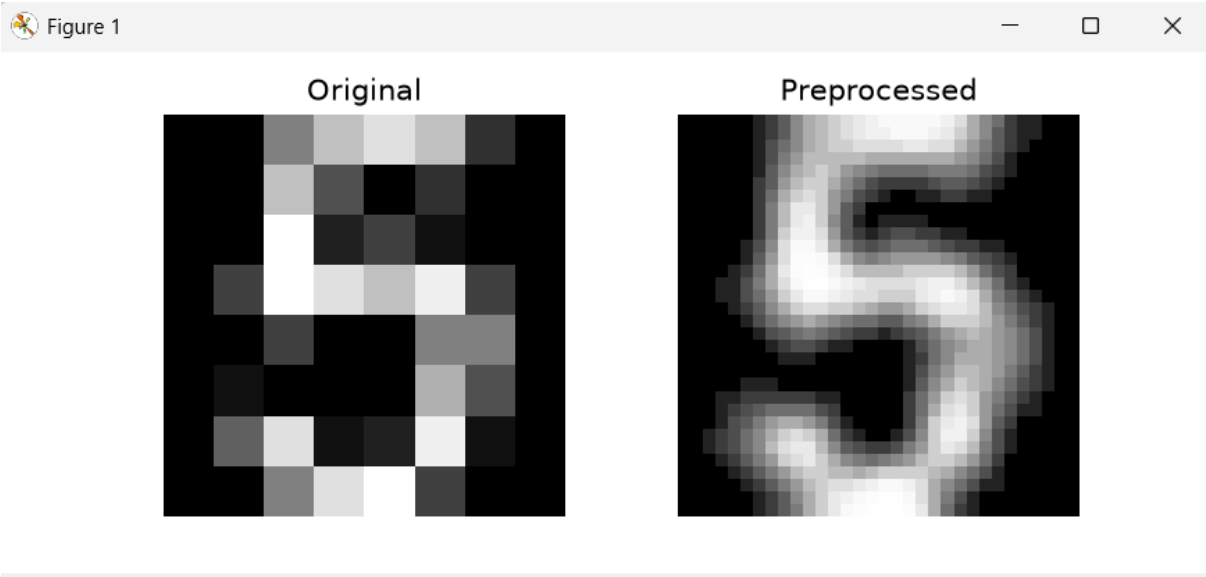
Accuracy: 99.72 %

Improvement: 2.22 percentage points

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/lenin/AppData/Local/Programs/Python/Python314/cv experiment
co3 at1.py
WITHOUT PREPROCESSING
Feature Size: 324
Accuracy: 97.5 %

WITH PREPROCESSING
Feature Size: 1764
Accuracy: 99.72 %

Improvement: 2.22 percentage points
```



Observation:

Parameter	**Without Preprocessing**	**With Preprocessing**
Feature extraction	HOG	HOG
Feature size	324	1764
Accuracy	97.50%	99.72%
Feature quality	Good	Better
Processing cost	Low	Higher

Analysis:

- Preprocessing improves image quality before feature extraction.
- HOG extracts more useful gradient information from the processed images.
- The preprocessed approach gives higher classification accuracy.
- The feature size and computational requirement also increase.

Result :

Feature extraction was successfully performed with and without preprocessing. The **preprocessed approach produced better feature representation and higher classification accuracy**.